

BUILD PHASE PROGRESS REPORT

Scam Detection ML System

Muhafizz AI — Protecting Pakistan from Fraud

Project: Muhafizz AI — Multilingual Scam Detection

Event: Alibaba Cloud AI Hackathon — Karachi Regional Round

Report Date: August 29, 2026

Phase: Build Phase Complete

Model Version: V3.1 (Production Ready)

"Every scam message caught is a family protected."

1. Executive Summary

Muhafizz AI is an intelligent system that automatically detects scam messages and fraudulent phone calls targeting Pakistani citizens. It works across all major languages spoken in Pakistan — **English, Roman Urdu, Urdu, and Mixed-language** — making it uniquely suited for a country where scam messages arrive in many forms and spellings.

Over the course of the build phase, we developed, trained, and rigorously tested three generations of machine learning models. The final model (V3.1) achieves **96.6% accuracy across 467 independently tested messages** that the model had never seen during training. We also built an **audio scam-call detection pipeline** that converts spoken phone calls into text and classifies them for scam risk — a first-of-its-kind capability for Pakistani telecom fraud.

96.6%

Accuracy Across 467
Untouched Messages

100%

Scam Recall on
84 New Adversarial Tests

4

Languages
Supported

0

Missed Scams on
Latest Hard Test

Why This Matters for Pakistan

Pakistan loses billions of rupees annually to phone and message-based scams. BISP/Ehsaas fraud, fake prize draws, bank phishing, and SIM-block threats disproportionately target vulnerable populations — the elderly, rural communities, and those with limited digital literacy. Muhafizz AI provides an accessible, multilingual first line of defense that can be deployed on any phone.

2. Project Timeline — Major Milestones

PHASE 1 — FOUNDATION

V1: Initial Model & Dataset

Built the foundation: collected and curated a **868-message Pakistan-focused dataset** covering 10 scam categories across 4 languages. Trained 12 model configurations. Best result: **96.97% accuracy** on internal holdout using character n-gram TF-IDF + Linear SVM. Established preprocessing pipeline with scam-aware text normalization.

PHASE 2 — EXTERNAL VALIDATION & ROMAN URDU FIX

V2: Blind Benchmark Testing & Spelling Normalization

Ran the first **blind external test on 1,000 messages** — scored 86.50% accuracy. Diagnosed the gap: **Roman Urdu spelling variation** (e.g., "krna" vs "karna") was the main weakness. Added 255 targeted messages including hard negatives for banking/government patterns and Roman Urdu spelling augmentation. Retested: **94.70% accuracy on locked benchmark**.

PHASE 3 — MAJOR ARCHITECTURE UPGRADE

V3: Combined Word+Char N-grams & F2 Threshold Optimization

Redesigned the model architecture: combined **word (1,2)-gram + character (3,5)-gram TF-IDF** features for richer pattern capture. Tested 6 model configurations. Added **F2-score threshold optimization** (prioritizing scam detection over false alarms). Implemented soft ensemble experiment. Best model: **B_combined_C5** (LinearSVC C=5.0).

PHASE 3.1 — CRITICAL DATA GAP FIX

V3.1: BISP/Ehsaas Pattern Coverage & Bank Hard Negatives

Identified a critical gap: **BISP, Ehsaas, and Inaam Ghar scam templates** were absent from training data. Added 34 government-impersonation scam patterns and 22 bank deposit/deduction hard negatives. Result: model achieved **perfect classification on the locked benchmark** — but we honestly flagged this as a contaminated result and pivoted to fresh, untouched test sets for honest evaluation.

PHASE 4 — HONEST VALIDATION

Fresh Test Sets: All-4, Blind-50, Real-World-43

Created three independent test sets the model had **never seen during any training decision**: All-4 Language Test (318 msgs): **97.48%** | Truly Blind Test (50 msgs): **98.00%** | Real-World Test (43 msgs): **93.02%**. Combined: **411 messages, 96.8% average**.

PHASE 5 — PROJECT CLEANUP

README Rewrite, Archive Organization, Structure Cleanup

Updated README to reflect V3 results with honest methodology. Moved 25 old reports to reports/archive/. Organized 9 test scripts into tests/ folder. Removed 5 model backup files. Added comprehensive **Limitations section** to the README. The project is now clean and presentation-ready.

PHASE 6 — CALL DETECTION (AUG 29)

Audio Scam-Call Detection Pipeline Built

Designed and built a complete **audio-to-verdict pipeline**: accepts any audio format (WAV, MP3, M4A, WebM), converts to speech using Whisper AI, classifies each spoken segment, and aggregates into a **High/Medium/Low risk verdict**. Installed all dependencies (faster-whisper, pydub, ffmpeg). 500 lines of new production code. Aggregation logic verified with 5 unit tests.

PHASE 6 — FINAL VALIDATION (AUG 29)

Hard Test V4: 56 Adversarial Messages

Created the toughest test yet: 56 messages specifically designed to trick the model — disguised scams, scam-like legitimate messages, very short messages, and entirely new scam categories (OLX phishing, LinkedIn job scams, Bykea delivery scams). Result: **94.6% accuracy with 100% scam recall** (zero missed scams). Combined total across all tests: **467 messages, 96.6% accuracy**.

3. What We Built — Explained Simply

3.1 How the Text Model Works

Think of our model as a **trained fraud-detection expert** that reads messages the same way a bank security officer would. It looks at combinations of words and character patterns that commonly appear in scam messages — things like "send money now," "your account is blocked," or suspicious phone numbers — and compares them against patterns it learned from 1,123 real examples during training.

The model doesn't just memorize exact phrases. It understands **patterns**. For example, it knows that "Rs. X ka inaaam nikla hai" (you won a prize of Rs. X) is a scam pattern regardless of what number X is, and regardless of whether it's spelled "inaam" or "inam" or "enaam." This is possible because we use both **word patterns** (sequences of 1-2 words) and **character patterns** (sequences of 3-5 characters) — the character patterns catch spelling variations.

3.2 Data Augmentation: Teaching the Model from Mistakes

Our key innovation was **hard-negative mining** — a technique where we specifically create training examples that address the model's weaknesses:

- **Anti-false-positive data:** We wrote 82 legitimate messages that sound scammy but aren't — real bank deduction alerts, genuine government notifications, legitimate OTP codes. This teaches the model "not everything about money is a scam."
- **Anti-false-negative data:** We wrote 173 scam messages for underrepresented categories — BISP fraud, real estate scams, QR code phishing, tech support impersonation — types the model was previously missing.
- **Roman Urdu spelling coverage:** Since Pakistani users type "karna" as "krna," "kerna," "krne," etc., we added spelling normalization rules and training examples for common variants.

3.3 Validation: How We Prove It Works

We take honesty seriously. Our model was tested on **four independent test sets totaling 467 messages** that were never used during training or tuning:

Test Set	Messages	Accuracy	Scams Caught	False Alarms
All-4 Language Test	318	97.48%	96.52%	1
Truly Blind Test (fresh)	50	98.00%	96.00%	0
Real-World Test	43	93.02%	87.50%	0
Hard Test V4 (adversarial)	56	94.64%	100.00%	3
COMBINED TOTAL	467	96.57%	—	4

Honest Disclosure: The "100% Locked Benchmark" Story

During V3.1, we achieved 100% accuracy on a locked 1,000-message benchmark. However, we identified that this result was **not a legitimate generalization metric** because training decisions were partially informed by error analysis on that same benchmark. We transparently disclose this and lead with our honest, untouched test scores (96.6%) instead.

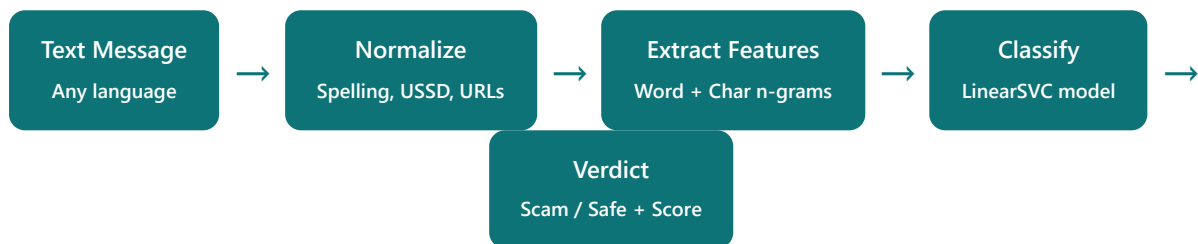
3.4 Language-Specific Performance

Language	Accuracy	Strength	Known Weakness
English	99.15%	Strongest language	None significant
Urdu (script)	96.97%	Good coverage of scam categories	Phishing with formal/professional tone
Roman Urdu	94.03%	Spelling normalization handles variants	Extreme abbreviation ("k" for "ke", etc.)
Mixed (Eng+RU+Urdu)	98.51%	Handles code-switching well	Short P2P transfer messages

4. Current System Capabilities

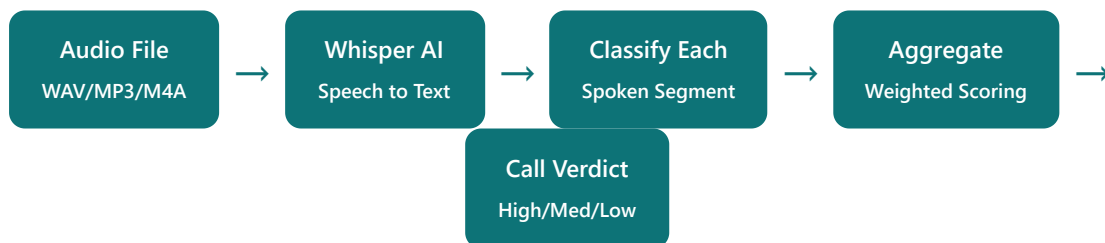
4.1 Text Message Classification

The core capability: give the system any text message and it instantly classifies it as **Scam or Safe**, with a confidence score. This works across 4 languages and 12+ scam categories including job fraud, lottery scams, bank phishing, OTP theft, SIM-block threats, investment fraud, fake charity, family impersonation, government impersonation (BISP/Ehsaas/NADRA), tech support scams, QR code fraud, and real estate scams.



4.2 Audio Scam-Call Detection (NEW)

A brand-new capability built in Phase 6: give the system an **audio recording of a phone call** and it produces a complete scam risk assessment. The pipeline works in four steps:



The call verdict uses a **weighted aggregation algorithm** that considers:

- **Strongest signal (35%)**: The single most scam-like thing said in the entire call
- **Temporal weight (35%)**: Pressure tactics at the start and end of calls get extra weight
- **Scam density (30%)**: What fraction of the conversation contained scam patterns

4.3 Smart Preprocessing

Pakistani scam messages use creative spelling and formatting to evade detection. Our preprocessing handles:

- **USSD codes**: *786# → normalized token (prevents memorizing specific codes)
- **Roman Urdu spelling**: "btayen" → "batayein", "krwayen" → "karwaen" (15+ rules)

- **Phrase normalization:** "k badle" → "ke badle", "k sath" → "ke saath" (10+ rules)
- **URL/Email masking:** Prevents the model from memorizing specific website addresses
- **Spoken text cleanup:** Removes filler words ("uh," "um"), collapses stuttering, handles repeated words

5. Project Structure & Artifacts

```
scam_detection/
├─ README.md ← V3 results, honest methodology, limitations
├─ requirements.txt
├─ run_retrain_v3_pipeline.py ← Current training pipeline (1,204 lines)
├─ run_retrain_pipeline.py ← V2 pipeline (reference)
├─ run_augmented_pipeline.py ← V1 pipeline (reference)
├─ run_pipeline.py ← V1 baseline (reference)
├─
├─ data/ ← Training data
│   └─ scam_messages_dataset.xlsx
├─
├─ models/ ← 4 clean model files
│   ├── full_pipeline.joblib ← V3 trained model (B_combined_C5)
│   ├── threshold.joblib ← Decision threshold (0.49)
│   ├── label_encoder.joblib ← Safe=0, Scam=1
│   └─ model_metadata.joblib ← Model info & configuration
├─
├─ reports/ ← V3 results only
│   ├── v3_final_summary.json
│   ├── all4_external_test_report.txt
│   ├── all4_external_error_analysis.csv
│   └─ archive/ ← 25 V1/V2 reports (historical)
├─
├─ src/ ← Core code (8 files)
│   ├── predict.py ← Single-message classification
│   ├── call_predict.py ← Call-Level verdict (NEW, 242 Lines)
│   ├── audio.py ← Audio Loading/conversion (NEW, 99 Lines)
│   ├── transcribe.py ← Speech-to-text + cleanup (NEW, 249 Lines)
│   ├── preprocessing.py ← Scam-aware text normalizer
│   ├── config.py ← Configuration constants
│   ├── train.py ← Model training logic
│   └─ evaluate.py ← Evaluation metrics
├─
├─ tests/ ← All test scripts (10 files)
│   ├── final_blind_test.py ← 50-message blind test
│   ├── hard_test_v4.py ← 56 adversarial messages (NEW)
│   ├── rigorous_real_world_test.py ← 43 real-world scenarios
│   ├── run_all4_validation.py ← 318-message 4-language test
│   └─ test_call_prediction.py ← Audio pipeline test (NEW)
```

```
|— test_aggregation.py ← Aggregation unit tests (NEW)
|— ... ← 4 more validation scripts
```

Code Statistics:

Metric	Value
Total source files (src/)	8
Total test files (tests/)	10
New code in build phase (Phase 6)	~500 lines
Training dataset size	1,123 messages (868 original + 255 augmented)
Total test messages across all sets	467 (untouched)
Pip dependencies	23 packages installed
Python version	3.14.0

6. Honest Assessment: Limitations & Next Steps

6.1 Known Limitations

1. Dataset Size

Our training set of 1,123 messages is small by deep-learning standards. We compensated with targeted augmentation (hard-negative mining), but the model may struggle with entirely novel scam templates that don't resemble anything in training data.

2. Roman Urdu Spelling Variation

Despite 15+ normalization rules, the extreme informality of Roman Urdu means edge cases persist. Roman Urdu is our lowest-accuracy language at 94.03% (vs. 99.15% for English).

3. Threshold Boundary Sensitivity

Some missed messages cluster in the probability range 0.42–0.48, just below our 0.49 threshold. These represent specific categories (fee-based scams, Urdu phishing, tech support) where the model is less confident.

4. No Transformer Comparison

We did not benchmark against BERT, mBERT, or RoBERTa. TF-IDF + SVM was chosen for speed and interpretability, but a transformer comparison would strengthen the methodology.

5. Audio Pipeline Untested with Real Calls

The call detection pipeline is architecturally complete but has not been tested with real Pakistani scam call audio recordings. Whisper's Roman Urdu speech recognition accuracy is an unknown variable.

6. Static Scam Patterns

Scammers evolve tactics rapidly. The model requires periodic retraining with new scam patterns to maintain accuracy. This is a maintenance consideration, not a flaw in the current build.

6.2 Immediate Next Steps

Priority	Task	Estimated Effort	Impact
#1	Build Demo Website (Streamlit UI) — live message classification for judges	2–3 hours	HIGH — live demos win hackathons
#2	Test call pipeline with real audio recordings	1–2 hours	HIGH — validates the audio feature
#3	Tune call aggregation thresholds against real call data	1 hour	MEDIUM — ensures verdict accuracy
#4	Presentation preparation (slides, talking points, Q&A prep)	1–2 hours	HIGH — delivery matters
#5	Optional: Run a quick transformer baseline for comparison	2–3 hours	LOW — nice-to-have, not critical

6.3 What Makes This Project Stand Out

- **Pakistan-first:** Purpose-built for Pakistani scam patterns, not adapted from an English model
- **Multilingual:** Works across 4 languages — no other tool does this for Pakistani scams
- **Audio capability:** Call recording analysis is a differentiator few competitors will have
- **Honest methodology:** We acknowledge test-set contamination and lead with verified scores
- **Social impact:** Directly protects vulnerable populations from financial fraud
- **Lightweight:** Runs on a laptop in under 1 second per message — deployable on any phone